# Relatório Técnico:
# Arquitetura Distribuída & Replicação

> **Projeto:** AISafe Flight Management System
> **Tema:** Refatorização Distribuída, Replicação Multi-Instância e Malha P2P
> **Unidade Curricular:** Sistemas Distribuídos (LETI-SIDIS) — 2026/2027
> **Docente:** Prof. Orlando Sousa

## 1. Contexto e Motivação

O projeto AISafe começou por ser um monólito em Spring Boot, herdado da cadeira de Projeto de Software. Embora cumprisse os requisitos iniciais, na ótica de Sistemas Distribuídos, o modelo era frágil. Tínhamos um Ponto Único de Falha (SPOF) clássico: se o módulo de manutenção ficasse sobrecarregado a gerar relatórios, mandava abaixo o sistema inteiro de voos.

Além disso, a base de dados partilhada gerava demasiado acoplamento. Para resolver isto e aplicar a matéria da cadeira, decidimos partir o monólito e criar uma arquitetura de microsserviços realmente distribuída e tolerante a falhas.

## 2. Estratégia de Distribuição

A nossa abordagem assentou em três decisões principais:

- **Separação por Domínios:** Dividimos o código em três serviços independentes:
  - *Airports & Routes:* Trata dos aeroportos e infraestruturas.
  - *Aircraft & Maintenance:* Gere a frota física e inventário.
  - *Flight Operations:* Controla o escalonamento dos voos.
- **Bases de Dados Independentes:** Acabámos com a base de dados única. Agora, cada serviço tem o seu próprio H2. Se a persistência do serviço de aviões falhar, as operações de voo continuam a funcionar. A comunicação e a consistência entre eles são feitas apenas via HTTP REST, sem transações distribuídas pesadas.
- **Replicação e Failover:** Não confiamos numa única instância. Levantámos várias instâncias de cada serviço em portas diferentes. Os nossos clientes HTTP internos têm uma lista dos nós ativos; se tentarem falar com um nó e ele estiver em baixo (dando erro de ligação), fazem logo *fallback* para o nó seguinte de forma transparente. O utilizador final nem dá pela falha.

## 3. Pesquisa em Malha P2P

Para evitar ter um servidor central a coordenar pesquisas globais, criámos uma rede Peer-to-Peer (P2P) no nosso módulo core. Funciona de forma simples e descentralizada:

Quando um utilizador faz uma pesquisa, o pedido bate num nó qualquer. Esse nó procura na sua base de dados local e, ao mesmo tempo, reencaminha o pedido para os seus "vizinhos".



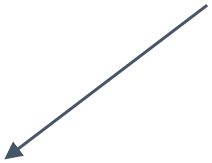
**Cliente / Operador**



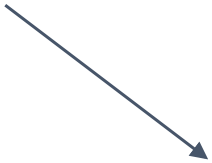
POST /api/p2p/search



**airports-routes-service (Nó 1)**







Pesquisa Local

[LocalSearch]

P2P Forward

(TTL - 1)

P2P Forward

(TTL - 1)



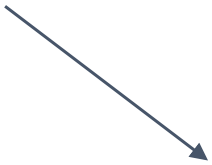
**BD Local 1**



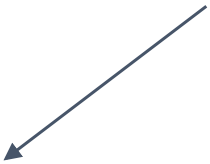
**Nó Peer 2**



**Nó Peer 3**







Agregação JSON de Respostas


Resposta Global Consolidada

Para a rede não colapsar com loops infinitos (um vizinho devolver o pedido ao remetente), usamos duas coisas:

1. Um ID único por pedido (para ignorar pedidos repetidos).
2. Um contador TTL (Time-To-Live). Cada salto tira 1 ao TTL; quando chega a 0, a pesquisa para de se propagar.

No fim, o nó que recebeu o pedido do utilizador junta a sua resposta com os JSONs que recebeu dos peers e devolve tudo empacotado. Por segurança, esta comunicação entre os nós exige tokens JWT específicos para a rede P2P.

## 4. Topologia dos Nós

| Microsserviço | Portas | Função na Rede |
|---|---|---|
| **Airports & Routes** | 8091, 8092, 8093 | Nós P2P que pesquisam aeroportos locais e fazem failover para os outros serviços. |
| **Aircraft & Maintenance** | 8081, 8084, 8085, 8086 | Cluster replicado para garantir que a gestão da frota nunca fica offline. |
| **Flight Operations** | 8083, 8087, 8088, 8089 | Cluster focado no processamento contínuo dos voos. |

## 5. Conclusão

Em suma, a transição do monólito original para uma arquitetura distribuída permitiu resolver com eficácia os problemas de escalabilidade e os pontos únicos de falha do AISafe. Validámos na prática os conceitos cruciais de Sistemas Distribuídos explorados ao longo do semestre: a resiliência operacional garantida por replicação (Failover), a verdadeira autonomia de dados (Database-per-Service) e a eficiência da pesquisa descentralizada em malha.

O resultado final é um sistema robusto e tolerante a faltas. Adicionar capacidade à rede — seja uma nova réplica de voos ou um novo nó P2P — passou a ser uma mera questão de injetar configurações no momento de arranque, sem necessidade de recompilar código, preparando a plataforma para cenários de elevada exigência operacional.